



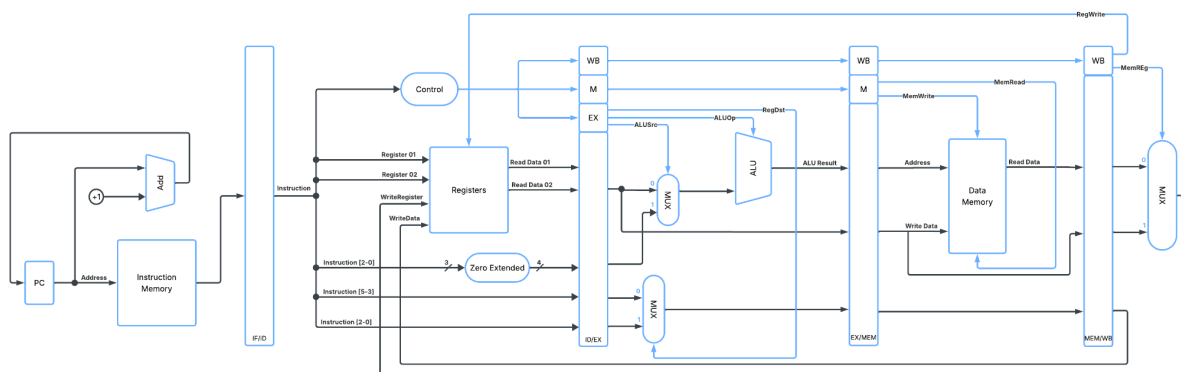
Relatório MIPS Pipeline

Implementação de um processador de 4 bits inspirado na arquitetura MIPS

Ana Luiza Boeng, Gabriele Bueno Santos, Marcello Eduardo Pereira Jorge e Mona Aya Kanso.

Esquemático da arquitetura

O processador possui 5 estágios, sendo eles o Instruction Fetch (1), Instruction Decode (2), Execute (3), Memory Access (4) e Write-Back (5), que terão a implementação detalhada ao decorrer do relatório. A arquitetura segue um modelo simplificado do MIPS Pipeline, sem conter o Hazard Detection Unit e o Forward Unit. Portanto, o processador não consegue lidar com hazard de dados e de controle, sendo necessário o uso da instrução NOP para evitar erros.



Esquemático da arquitetura do processador

Registradores de Pipeline

Foram implementados os registradores IF/ID, ID/EX, EX/MEM e MEM/WB para realizar a transferência das informações entre os estágios do processador.

O registrador IF/ID armazena a instrução buscada na memória, disponibilizando essa informação para o estágio de decodificação. Como não há instruções que utilizem o valor do Program Counter fora do primeiro estágio, não houve necessidade de armazená-lo ou passá-lo para os próximos estágios.

O registrador ID/EX armazena os dados lidos do banco de registradores, os valores imediatos e os sinais de controle gerados pela Unidade de Controle, encaminhando-os para o estágio de execução.

O registrador EX/MEM armazena os resultados produzidos pela ULA e os sinais de controle necessários para o acesso à memória de dados.

Por fim, o registrador MEM/WB armazena os dados provenientes da memória ou da ULA, disponibilizando-os para escrita no banco de registradores durante o estágio Write-Back.

Todos os registradores foram implementados utilizando processos síncronos ao clock, realizando a captura dos sinais na borda de subida e mantendo os valores armazenados até o próximo ciclo.

Estágio IF (Instruction Fetch)

Visto que o conjunto de instruções implementado não possui instruções de desvio (branch ou jump), não houve necessidade da utilização de um multiplexador (MUX) para seleção do próximo valor do Program Counter (PC). Dessa forma, o PC apenas incrementa seu valor a cada ciclo de clock, realizando a busca sequencial das instruções armazenadas na memória.

O estágio IF foi implementado utilizando os seguintes componentes:

- Program Counter (PC);
- Incrementador;
- Memória ROM de instruções;
- Registrador de pipeline IF/ID.

Para verificar o funcionamento do estágio, foi desenvolvido um testbench isolado. O objetivo do teste foi validar o incremento correto do PC, o acesso à memória ROM e o armazenamento das instruções no registrador IF/ID.

Foram inseridas instruções de teste manualmente na memória ROM para facilitar a visualização e a validação dos resultados. A simulação foi realizada utilizando o simulador ISim da ferramenta Xilinx ISE.

Os resultados obtidos demonstraram o funcionamento correto do estágio IF. Observou-se que o PC foi incrementado sequencialmente a cada ciclo de clock, acessando corretamente as posições da memória ROM. Além disso, verificou-se que as instruções lidas foram armazenadas pelo registrador IF/ID e disponibilizadas para o próximo estágio do pipeline com atraso de um ciclo de clock.

A seguir são apresentados a memória ROM utilizada no teste, o gráfico de simulação e o relatório gerado pelo simulador.

Conteúdo da Memória ROM utilizada

A memória Rom possui instruções do tipo R e do tipo I, que seguem o seguinte formato:

Instruções tipo R:

OPCODE	RS	RT	RD
11			0

Instruções tipo I:

OPCODE	RS	RD	IMM
11			0

Sendo que os registradores do banco de registradores foram endereçados da seguinte forma:

Registrador	Endereço	Descrição
R0	000	Registrador com valor zero
R1	001	Registrador que armazena o resultado da instrução ADD
R2	010	Registrador que armazena o resultado da instrução SUB
R3	011	Registrador não utilizado
R4	100	Registrador contendo o valor 4
R5	101	Registrador contendo o valor 5
R6	110	Registrador não utilizado
SP	111	Registrador que aponta para a memória de dados

E os códigos de operação de cada instrução foram divididos da seguinte forma:

Instrução	OPCODE
NOP	000
ADD	001
SUB	010
AND	011
LOAD WORD	100
STORE WORD	101
ADDI	110

Portanto, as instruções da memória, seguindo os códigos e endereços mostrados anteriormente, foram montados da seguinte forma:

0 => "110000100100" -- 0: ADDI R0(ZERO), R4, 4
1 => "110000101101" -- 1: ADDI R0, R5, 5

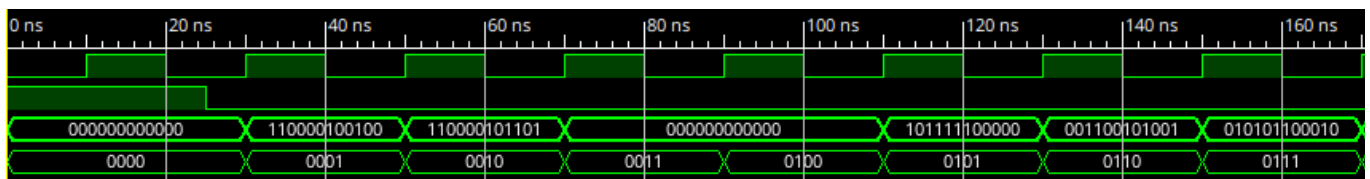
```

2 => "000000000000" -- 2: NOP
3 => "000000000000" -- 3: NOP
4 => "101111100000" -- 4: SW SP, R4, 0(SP)
5 => "001100101001" -- 5: ADD R4, R5, R1
6 => "010101100010" -- 6: SUB R5, R4, R2
7 => "100111001000" -- 7: LW SP, R1, 0(SP)

```

Observe que a escrita das instruções é feita de acordo com o endereçamento binário do opcode, registradores e imediato.

Gráfico de simulação



Relatório do simulador

at 25 ns: Note: Reset finalizado (/tb_datapath_if/).

at 31 ns: Note: PC = 0001 (1) | IF_ID = 110000100100 (/tb_datapath_if/).

at 51 ns: Note: PC = 0010 (2) | IF_ID = 110000101101 (/tb_datapath_if/).

at 71 ns: Note: PC = 0011 (3) | IF_ID = 000000000000 (/tb_datapath_if/).

at 91 ns: Note: PC = 0100 (4) | IF_ID = 000000000000 (/tb_datapath_if/).

at 111 ns: Note: PC = 0101 (5) | IF_ID = 101111100000 (/tb_datapath_if/).

at 131 ns: Note: PC = 0110 (6) | IF_ID = 001100101001 (/tb_datapath_if/).

at 151 ns: Note: PC = 0111 (7) | IF_ID = 010101100010 (/tb_datapath_if/).

at 171 ns: Note: PC = 1000 (8) | IF_ID = 100111001000 (/tb_datapath_if/).

at 191 ns: Note: PC = 1001 (9) | IF_ID = 000000000000 (/tb_datapath_if/).

at 211 ns: Note: PC = 1010 (10) | IF_ID = 000000000000 (/tb_datapath_if/).

at 231 ns: Note: PC = 1011 (11) | IF_ID = 000000000000 (/tb_datapath_if/).

at 251 ns: Note: PC = 1100 (12) | IF_ID = 000000000000 (/tb_datapath_if/).

at 271 ns: Note: PC = 1101 (13) | IF_ID = 000000000000 (/tb_datapath_if/).

at 291 ns: Note: PC = 1110 (14) | IF_ID = 000000000000 (/tb_datapath_if/).

at 311 ns: Note: PC = 1111 (15) | IF_ID = 000000000000 (/tb_datapath_if/).

at 331 ns: Note: PC = 0000 (0) | IF_ID = 000000000000 (/tb_datapath_if/).

at 351 ns: Note: PC = 0001 (1) | IF_ID = 110000100100 (/tb_datapath_if/).

at 351 ns: Note: Fim da simulacao do datapath_if (/tb_datapath_if/).

Estágio ID (Instruction Decode)

O Estágio de decodificação (ID) é responsável por disponibilizar os valores armazenados no banco de registradores para os próximos estágios, aumentar a quantidade de bits do valor imediato e gerar sinais de controle para todos os estágios.

O estágio ID foi implementado agrupando os seguintes componentes:

- Zero Extended: recebe um vetor de 3 bits e converte para um vetor de 4 bits.
- Unidade de Controle: recebe o OP CODE da instrução e gera os sinais de controle necessários para cada estágio.
- Banco de registradores: recebe os endereços de leitura e escrita e disponibiliza os dados de leitura na saída do banco. Recebe também um valor a ser escrito, e na descida do clock, caso o sinal de controle “RegWrite” esteja ativo, escreve o valor no registrador correspondente.

Segue abaixo a tabela de sinais de controle de cada operação:

Sinais de Controle	INSTRUÇÕES			
	Operações lógicas e aritméticas	ADDI	LW	SW
RegDst	0	1	1	1
RegWrite	1	1	1	0
AluSrc	0	1	1	1
ALUOP	*	010	010	010
MemWrite	0	0	0	1
MemRead	0	0	1	0
MemtoReg	0	0	1	0

* Cada operação aritmética tem seu código de operação da ULA, sendo “010” para soma, “110” para subtração e e “000” para operação lógica AND.

Estágio EX (Execute)

O estágio de Execução (EX) atua como o núcleo do processador pipeline, sendo responsável por processar as operações aritméticas e lógicas, além de calcular antecipadamente os endereços físicos. Neste projeto de pipeline, este estágio opera de maneira estritamente combinacional, processando os dados brutos estabilizados no registrador de barreira ID/EX.

O estágio EX foi implementado agrupando os seguintes componentes no arquivo estrutural principal (datapath_ex):

- **Unidade Lógica Aritmética (ULA):** Bloco combinacional genérico parametrizado para 4 bits de dados, responsável por executar as operações aritméticas e lógicas da arquitetura, utilizando um sinal de controle de 3 bits para selecionar a operação desejada.
- **Multiplexadores (MUX):** Módulos parametrizáveis utilizados para o roteamento correto dos dados. O MUX *ALUSrc* define se o segundo operando da ULA virá do banco de registradores (Read Data 2) ou do valor imediato estendido. O MUX *RegDst* seleciona qual campo da instrução apontará para o registrador de destino da escrita



(RT ou RD).

Simulação e Validação

Para verificar a integridade do roteamento interno e o funcionamento aritmético, foi desenvolvido um *testbench* isolado (*tb_datapath_ex*) que injetou sinais de controle diretamente nas portas de entrada do módulo, simulando os dados provenientes do estágio ID. Foram validados três cenários principais de transição de estado:

1. **Operação Tipo R (ADD):** O MUX *RegDst* roteou corretamente o destino para o registrador RD, enquanto a ULA processou a soma exata dos valores advindos de *Read Data 1* e *Read Data 2*.
2. **Acesso à Memória (LW):** O MUX *ALUSrc* chaveou perfeitamente a entrada da ULA para receber o imediato com extensão de sinal, executando a soma com o registrador base para gerar o endereço correto de memória para o próximo estágio.

Estágio MEM (Memory Access)

O estágio MEM foi implementado no módulo *datapath_mem*, responsável por integrar a memória de dados e o registrador de pipeline MEM/WB.

O módulo recebe do registrador EX/MEM o resultado da ULA, o dado a ser escrito na memória, o registrador de destino e os sinais de controle necessários para as operações de memória. O resultado da ULA é utilizado como endereço de acesso à memória de dados.

A memória de dados foi implementada no módulo *memoria_dados*, utilizando uma memória RAM simples de 16 posições, com palavras de 4 bits. Quando o sinal *MemWrite* está ativo, o valor recebido em *write_data* é armazenado na posição indicada pelo endereço. Quando o sinal *MemRead* está ativo, o conteúdo da posição acessada é disponibilizado na saída da memória.

Após o acesso à memória, o registrador MEM/WB armazena o dado lido da memória, o resultado da ULA, o registrador de destino e os sinais de controle *RegWrite* e *MemtoReg*.

Esses sinais são encaminhados para o estágio WB, onde será definido o valor a ser escrito no banco de registradores.

Dessa forma, o estágio MEM realiza o acesso à memória de dados e encaminha as informações necessárias para a etapa final de escrita do processador.

Estágio WB (Write-Back)

O estágio WB foi implementado no módulo `datapath_wb`, responsável por selecionar o dado que será escrito no banco de registradores ao final da execução da instrução.

O módulo recebe os dados provenientes do registrador MEM/WB, sendo eles o resultado da ULA, o dado lido da memória e o registrador de destino. A seleção entre o resultado da ULA e o dado da memória é feita por um multiplexador controlado pelo sinal `MemtoReg`.

Quando `MemtoReg` está em nível lógico baixo, o valor encaminhado para escrita é o resultado da ULA, utilizado em instruções aritméticas. Quando `MemtoReg` está em nível lógico alto, o valor encaminhado é o dado lido da memória, utilizado em instruções do tipo LW.

Além da seleção do dado, o estágio WB também repassa o registrador de destino e o sinal `RegWrite` para o banco de registradores. Quando `RegWrite` está ativo, o valor selecionado é escrito no registrador correspondente.

Dessa forma, o estágio WB finaliza a execução da instrução, disponibilizando ao banco de registradores o dado correto, o endereço do registrador de destino e o sinal de habilitação de escrita.

Testes Finais

Após a integração dos estágios do processador, alguns testes foram realizados utilizando o arquivo `"tb_pipeline_debug"`, responsável por exibir os sinais internos dos estágios IF, ID, EX, MEM e WB. Nos dois primeiros ciclos da simulação, o sinal de *reset* permanece ativo para inicializar o processador. Sendo assim, a execução das instruções começa, de fato, a partir do terceiro ciclo de clock.

Observe que durante a análise dos relatórios do ISim, o valor do Program Counter exibido aparece uma posição à frente da instrução buscada.

Teste 01: Instruções Aritméticas e Lógicas

O primeiro teste teve como objetivo verificar o funcionamento das instruções ADDI, ADD, SUB e AND. Inicialmente, as instruções ADDI carregam os valores 4 e 5 nos registradores R4 e R5. Em seguida, as instruções aritméticas e lógica utilizam esses valores para produzir os resultados em R1, R2 e R3.

Instruções utilizadas:

```
0 => "110000100100", -- ADDI R0, R4, 4
1 => "110000101101", -- ADDI R0, R5, 5
2 => "000000000000", -- NOP
3 => "000000000000", -- NOP
4 => "001100101001", -- ADD R4, R5, R1
5 => "010101100010", -- SUB R5, R4, R2
6 => "011100101011", -- AND R4, R5, R3
7 => "000000000000", -- NOP
8 => "000000000000", -- NOP
9 => "000000000000", -- NOP
```

Resultado esperado:

R4 = 4

R5 = 5

R1 = R4 + R5 = 9

R2 = R5 - R4 = 1

R3 = R4 AND R5 = 4

No estágio IF, as instruções foram buscadas na ordem definida dentro da ROM. No estágio ID, os sinais de controle foram gerados corretamente: ADDI, por exemplo, utilizou *alu_src* = 1, pois usa um valor imediato, enquanto ADD, SUB e AND utilizaram *alu_src* = 0, pois usam dois registradores. No estágio EX, a ULA realizou as operações esperadas. Por fim, no estágio WB, os resultados foram escritos nos registradores corretos.

As escritas observadas no estágio WB foram:

R4 = 0100

R5 = 0101

R1 = 1001

R2 = 0001

R3 = 0100

A Figura abaixo apresenta os ciclos finais do teste, nos quais é possível observar a escrita dos resultados das operações ADD, SUB e AND no estágio WB.

```

at 96 ns: Note: Ciclo 10 | reset='0' (/tb_pipeline_debug/).
at 96 ns: Note: IF | pc=1000 | instr=000000000000 (/tb_pipeline_debug/).
at 96 ns: Note: ID | acc=0100 | reg=0101 | imm=0011 | rt=101 | rd=011 (/tb_pipeline_debug/).
at 96 ns: Note: ID | reg_dst='1' | reg_write='1' | alu_src='0' | alu_ctrl=011 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: EX | alu_result=0001 | dado_mem=0100 | reg_dest=010 (/tb_pipeline_debug/).
at 96 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: MEM | dado_mem=0000 | alu_result=1001 | reg_dest=001 (/tb_pipeline_debug/).
at 96 ns: Note: MEM | reg_write='1' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: WB | write_data=1001 | write_reg=001 | reg_write='1' (/tb_pipeline_debug/).
at 106 ns: Note: ===== (/tb_pipeline_debug/).
at 106 ns: Note: Ciclo 11 | reset='0' (/tb_pipeline_debug/).
at 106 ns: Note: IF | pc=1001 | instr=000000000000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: EX | alu_result=0100 | dado_mem=0101 | reg_dest=011 (/tb_pipeline_debug/).
at 106 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: MEM | dado_mem=0000 | alu_result=0001 | reg_dest=010 (/tb_pipeline_debug/).
at 106 ns: Note: MEM | reg_write='1' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: WB | write_data=0001 | write_reg=010 | reg_write='1' (/tb_pipeline_debug/).
at 116 ns: Note: ===== (/tb_pipeline_debug/).
at 116 ns: Note: Ciclo 12 | reset='0' (/tb_pipeline_debug/).
at 116 ns: Note: IF | pc=1010 | instr=000000000000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 116 ns: Note: EX | reg_write='0' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: MEM | dado_mem=0000 | alu_result=0100 | reg_dest=011 (/tb_pipeline_debug/).
at 116 ns: Note: MEM | reg_write='1' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: WB | write_data=0100 | write_reg=011 | reg_write='1' (/tb_pipeline_debug/).

```

Com isso, o Teste 01 confirmou o funcionamento correto das instruções aritméticas e lógicas, além da escrita dos resultados no banco de registradores.

Teste 02: Instruções de Memória

O segundo teste teve como objetivo verificar as instruções de acesso à memória, SW e LW. Primeiro, os valores 4 e 5 foram carregados em R4 e R5. Depois, esses valores foram armazenados na memória de dados. Por fim, foram carregados novamente para R1 e R2.

Instruções utilizadas:

```

0 => "110000100100", -- ADDI R0, R4, 4
1 => "110000101101", -- ADDI R0, R5, 5
2 => "000000000000", -- NOP
3 => "000000000000", -- NOP
4 => "101111100000", -- SW SP, R4, 0(SP)
5 => "101111101001", -- SW SP, R5, 1(SP)
6 => "100111001000", -- LW SP, R1, 0(SP)
7 => "100111010001", -- LW SP, R2, 1(SP)

```

Resultado esperado:

R4 = 4

R5 = 5

Mem_de_dados[0] = R4 = 4

Mem_de_dados[1] = R5 = 5

R1 = Mem_de_dados[0] = 4

R2 = Mem_de_dados[1] = 5

No estágio IF, as instruções foram buscadas corretamente. No estágio ID, as instruções SW ativaram o sinal *mem_write*, indicando escrita na memória, enquanto as instruções LW ativaram *mem_read*, *mem_to_reg* e *reg_write*, indicando leitura da memória e escrita no banco de registradores. No estágio EX, a ULA calculou os endereços de memória usando SP + deslocamento. No estágio MEM, ocorreram as escritas e leituras na memória. No estágio WB, os valores lidos foram escritos em R1 e R2.

Os resultados observados foram:

R4 = 0100

R5 = 0101

R1 = 0100

R2 = 0101

A imagem abaixo ilustra a execução das instruções SW, com o sinal *mem_write* ativado e os valores de R4 e R5 sendo enviados para a memória de dados.

```
at 86 ns: Note: Ciclo 9 | reset='0' (/tb_pipeline_debug/).
at 86 ns: Note: IF | pc=0111 | instr=100111001000 (/tb_pipeline_debug/).
at 86 ns: Note: ID | acc=0000 | reg=0101 | imm=0001 | rt=101 | rd=001 (/tb_pipeline_debug/).
at 86 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='1' | alu_ctrl=010 | mem_write='1' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 86 ns: Note: EX | alu_result=0000 | dado_mem=0100 | reg_dest=100 (/tb_pipeline_debug/).
at 86 ns: Note: EX | reg_write='0' | mem_write='1' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 86 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 86 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 86 ns: Note: WB | write_data=0000 | write_reg=000 | reg_write='0' (/tb_pipeline_debug/).
at 96 ns: Note: ===== (/tb_pipeline_debug/).
at 96 ns: Note: Ciclo 10 | reset='0' (/tb_pipeline_debug/).
at 96 ns: Note: IF | pc=1000 | instr=100111010001 (/tb_pipeline_debug/).
at 96 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=001 | rd=000 (/tb_pipeline_debug/).
at 96 ns: Note: ID | reg_dst='0' | reg_write='1' | alu_src='1' | alu_ctrl=010 | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 96 ns: Note: EX | alu_result=0001 | dado_mem=0101 | reg_dest=101 (/tb_pipeline_debug/).
at 96 ns: Note: EX | reg_write='0' | mem_write='1' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=100 (/tb_pipeline_debug/).
at 96 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: WB | write_data=0000 | write_reg=100 | reg_write='0' (/tb_pipeline_debug/).
at 106 ns: Note: ===== (/tb_pipeline_debug/).
at 106 ns: Note: Ciclo 11 | reset='0' (/tb_pipeline_debug/).
at 106 ns: Note: IF | pc=1001 | instr=000000000000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | acc=0000 | reg=0000 | imm=0001 | rt=010 | rd=001 (/tb_pipeline_debug/).
at 106 ns: Note: ID | reg_dst='0' | reg_write='1' | alu_src='1' | alu_ctrl=010 | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 106 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=001 (/tb_pipeline_debug/).
at 106 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 106 ns: Note: MEM | dado_mem=0000 | alu_result=0001 | reg_dest=101 (/tb_pipeline_debug/).
at 106 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: WB | write_data=0001 | write_reg=101 | reg_write='0' (/tb_pipeline_debug/).
at 116 ns: Note: ===== (/tb_pipeline_debug/).
```

Já a próxima imagem mostra as instruções LW carregando os valores da memória e escrevendo-os nos registradores R1 e R2.

```

at 116 ns: Note: Ciclo 12 | reset='0' (/tb_pipeline_debug/).
at 116 ns: Note: IF | pc=1010 | instr=000000000000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: EX | alu_result=0001 | dado_mem=0000 | reg_dest=010 (/tb_pipeline_debug/).
at 116 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 116 ns: Note: MEM | dado_mem=0100 | alu_result=0000 | reg_dest=001 (/tb_pipeline_debug/).
at 116 ns: Note: MEM | reg_write='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 116 ns: Note: WB | write_data=0100 | write_reg=001 | reg_write='1' (/tb_pipeline_debug/).
at 126 ns: Note: ===== (/tb_pipeline_debug/).
at 126 ns: Note: Ciclo 13 | reset='0' (/tb_pipeline_debug/).
at 126 ns: Note: IF | pc=1011 | instr=000000000000 (/tb_pipeline_debug/).
at 126 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 126 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 126 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 126 ns: Note: EX | reg_write='0' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 126 ns: Note: MEM | dado_mem=0101 | alu_result=0001 | reg_dest=010 (/tb_pipeline_debug/).
at 126 ns: Note: MEM | reg_write='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 126 ns: Note: WB | write_data=0101 | write_reg=010 | reg_write='1' (/tb_pipeline_debug/).

```

Assim, o Teste 02 confirmou que os valores foram armazenados corretamente na memória e carregados de volta para o banco de registradores.

Teste 03: Instruções “Mistas”

O terceiro teste teve como objetivo verificar o funcionamento integrado entre operações aritméticas e de acesso à memória. Nesse teste, foram utilizadas instruções ADDI, SW, ADD, SUB e LW.

O programa carrega $R4 = 4$ e $R5 = 5$, armazena $R4$ na memória, executa uma soma e uma subtração e, por fim, carrega novamente o valor salvo na memória para $R1$.

Instruções utilizadas:

```

0 => "110000100100", -- 0: ADDI R0, R4, 4
1 => "110000101101", -- 1: ADDI R0, R5, 5
2 => "000000000000", -- 2: NOP
3 => "000000000000", -- 3: NOP
4 => "101111100000", -- 4: SW SP, R4, 0(SP)
5 => "001100101001", -- 5: ADD R4, R5, R1
6 => "010101100010", -- 6: SUB R5, R4, R2
7 => "100111001000", -- 7: LW SP, R1, 0(SP)

```

Resultado esperado:

$R4 = 4$

$R5 = 5$

$\text{Mem_de_dados}[0] = R4 = 4$

$R1 = R4 + R5 = 9$

$R2 = R5 - R4 = 1$

$R1 = \text{Mem_de_dados}[0] = 4$

Esse teste mostra que o registrador $R1$ recebe primeiro o resultado da instrução ADD, ou seja, 1001, correspondente a 9. Porém, posteriormente, a instrução LW também utiliza $R1$

como destino. Por isso, o valor anterior é sobrescrito, e o valor final de R1 passa a ser 0100, 4 em decimal.

Nos estágios do pipeline, a execução ocorreu como esperado. A ULA foi utilizada tanto para operações aritméticas quanto para cálculo de endereço de memória. A memória recebeu corretamente o valor de R4, e o estágio WB escreveu os resultados finais nos registradores de destino.

Resultados observados:

R4 = 0100

R5 = 0101

Mem_de_dados[0] = 0100

R1 após ADD = 1001

R2 = 0001

R1 após LW = 0100

A figura abaixo apresenta a parte aritmética do teste misto, mostrando a escrita de R1 = 1001 após o ADD e de R2 = 0001 após o SUB.

```
at 96 ns: Note: Ciclo 10 | reset='0' (/tb_pipeline_debug/).
at 96 ns: Note: IF | pc=1000 | instr=100111001000 (/tb_pipeline_debug/).
at 96 ns: Note: ID | acc=0101 | reg=0100 | imm=0010 | rt=100 | rd=010 (/tb_pipeline_debug/).
at 96 ns: Note: ID | reg_dst='1' | reg_write='1' | alu_src='0' | alu_ctrl=110 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: EX | alu_result=1001 | dado_mem=0101 | reg_dest=001 (/tb_pipeline_debug/).
at 96 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=100 (/tb_pipeline_debug/).
at 96 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: WB | write_data=0000 | write_reg=100 | reg_write='0' (/tb_pipeline_debug/).
at 106 ns: Note: ===== (/tb_pipeline_debug/).
at 106 ns: Note: Ciclo 11 | reset='0' (/tb_pipeline_debug/).
at 106 ns: Note: IF | pc=1001 | instr=000000000000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=001 | rd=000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | reg_dst='0' | reg_write='1' | alu_src='1' | alu_ctrl=010 | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 106 ns: Note: EX | alu_result=0001 | dado_mem=0100 | reg_dest=010 (/tb_pipeline_debug/).
at 106 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: MEM | dado_mem=0000 | alu_result=1001 | reg_dest=001 (/tb_pipeline_debug/).
at 106 ns: Note: MEM | reg_write='1' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: WB | write_data=1001 | write_reg=001 | reg_write='1' (/tb_pipeline_debug/).
at 116 ns: Note: ===== (/tb_pipeline_debug/).
at 116 ns: Note: Ciclo 12 | reset='0' (/tb_pipeline_debug/).
at 116 ns: Note: IF | pc=1010 | instr=000000000000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=001 (/tb_pipeline_debug/).
at 116 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 116 ns: Note: MEM | dado_mem=0000 | alu_result=0001 | reg_dest=010 (/tb_pipeline_debug/).
at 116 ns: Note: MEM | reg_write='1' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: WB | write_data=0001 | write_reg=010 | reg_write='1' (/tb_pipeline_debug/).
at 126 ns: Note: ===== (/tb_pipeline_debug/).
```

Enquanto esta imagem mostra a instrução LW carregando o valor armazenado na memória e sobrescrevendo R1 com 0100.

```
at 126 ns: Note: Ciclo 13 | reset='0' (/tb_pipeline_debug/).
at 126 ns: Note: IF | pc=1011 | instr=000000000000 (/tb_pipeline_debug/).
at 126 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 126 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 126 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 126 ns: Note: EX | reg_write='0' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 126 ns: Note: MEM | dado_mem=0100 | alu_result=0000 | reg_dest=001 (/tb_pipeline_debug/).
at 126 ns: Note: MEM | reg_write='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 126 ns: Note: WB | write_data=0100 | write_reg=001 | reg_write='1' (/tb_pipeline_debug/).
```

Dessa forma, o Teste 03 confirmou a integração entre banco de registradores, ULA, memória de dados e estágio de escrita.

Teste 04: Demonstração de Hazard de Dados

O quarto teste teve como objetivo demonstrar a ocorrência de um hazard de dados. Para isso, foi removido propositalmente o intervalo de instruções NOP entre uma instrução que produz um valor e outra que utiliza esse mesmo valor logo em seguida.

Nesse teste, a instrução ADD R4, R5, R1 calcula o valor 9 e deveria armazená-lo em R1. Logo em seguida, a instrução SW SP, R1, 0(SP) deveria salvar esse valor na memória. Porém, como o processador não possui forwarding nem detecção automática de hazards, o SW lê R1 antes que o ADD termine de escrever o resultado no banco de registradores.

Instruções utilizadas:

```
0 => "110000100100", -- ADDI R0, R4, 4
1 => "110000101101", -- ADDI R0, R5, 5
2 => "000000000000", -- NOP
3 => "000000000000", -- NOP
4 => "001100101001", -- ADD R4, R5, R1
5 => "101111001000", -- SW SP, R1, 0(SP)
6 => "100111010000", -- LW SP, R2, 0(SP)
```

Resultado esperado sem considerar hazard:

R1 = 9

Mem_de_dados[0] = 9

R2 = Mem_de_dados[0] = 9

Resultado obtido com hazard:

R1 = 9

Mem_de_dados[0] = 0

R2 = 0

No pipeline, o ADD calcula corretamente 1001 no estágio EX e depois escreve esse valor em R1. Entretanto, a instrução SW já havia lido o valor antigo de R1 no estágio ID, antes da escrita do ADD. Assim, o valor armazenado na memória foi 0000, e não 1001. Em seguida, o LW carregou esse valor incorreto da memória para R2.

A próxima figura ilustra o hazard de dados: o ADD calcula corretamente o valor 1001, mas o SW utiliza o valor antigo de R1, ainda igual a 0000.

```

at 76 ns: Note: Ciclo 8 | reset='0' (/tb_pipeline_debug/).
at 76 ns: Note: IF | pc=0110 | instr=101111001000 (/tb_pipeline_debug/).
at 76 ns: Note: ID | acc=0100 | reg=0101 | imm=0001 | rt=101 | rd=001 (/tb_pipeline_debug/).
at 76 ns: Note: ID | reg_dst='1' | reg_write='1' | alu_src='0' | alu_ctrl=010 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 76 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 76 ns: Note: EX | reg_write='0' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 76 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 76 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 76 ns: Note: WB | write_data=0000 | write_reg=000 | reg_write='0' (/tb_pipeline_debug/).
at 86 ns: Note: ===== (/tb_pipeline_debug/).
at 86 ns: Note: Ciclo 9 | reset='0' (/tb_pipeline_debug/).
at 86 ns: Note: IF | pc=0111 | instr=100111010000 (/tb_pipeline_debug/).
at 86 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=001 | rd=000 (/tb_pipeline_debug/).
at 86 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='1' | alu_ctrl=010 | mem_write='1' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 86 ns: Note: EX | alu_result=1001 | dado_mem=0101 | reg_dest=001 (/tb_pipeline_debug/).
at 86 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 86 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 86 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 86 ns: Note: WB | write_data=0000 | write_reg=000 | reg_write='0' (/tb_pipeline_debug/).
at 96 ns: Note: ===== (/tb_pipeline_debug/).
at 96 ns: Note: Ciclo 10 | reset='0' (/tb_pipeline_debug/).
at 96 ns: Note: IF | pc=1000 | instr=000000000000 (/tb_pipeline_debug/).
at 96 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=010 | rd=000 (/tb_pipeline_debug/).
at 96 ns: Note: ID | reg_dst='0' | reg_write='1' | alu_src='1' | alu_ctrl=010 | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 96 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=001 (/tb_pipeline_debug/).
at 96 ns: Note: EX | reg_write='0' | mem_write='1' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: MEM | dado_mem=0000 | alu_result=1001 | reg_dest=001 (/tb_pipeline_debug/).
at 96 ns: Note: MEM | reg_write='1' | mem_to_reg='0' (/tb_pipeline_debug/).
at 96 ns: Note: WB | write_data=1001 | write_reg=001 | reg_write='1' (/tb_pipeline_debug/).
at 106 ns: Note: ===== (/tb_pipeline_debug/).

```

Mostrando que o LW carrega 0000 da memória para R2, temos a imagem abaixo confirmando a informação:

```

at 106 ns: Note: Ciclo 11 | reset='0' (/tb_pipeline_debug/).
at 106 ns: Note: IF | pc=1001 | instr=000000000000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 106 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=010 (/tb_pipeline_debug/).
at 106 ns: Note: EX | reg_write='1' | mem_write='0' | mem_read='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 106 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=001 (/tb_pipeline_debug/).
at 106 ns: Note: MEM | reg_write='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 106 ns: Note: WB | write_data=0000 | write_reg=001 | reg_write='0' (/tb_pipeline_debug/).
at 116 ns: Note: ===== (/tb_pipeline_debug/).
at 116 ns: Note: Ciclo 12 | reset='0' (/tb_pipeline_debug/).
at 116 ns: Note: IF | pc=1010 | instr=000000000000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | acc=0000 | reg=0000 | imm=0000 | rt=000 | rd=000 (/tb_pipeline_debug/).
at 116 ns: Note: ID | reg_dst='0' | reg_write='0' | alu_src='0' | alu_ctrl=000 | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: EX | alu_result=0000 | dado_mem=0000 | reg_dest=000 (/tb_pipeline_debug/).
at 116 ns: Note: EX | reg_write='0' | mem_write='0' | mem_read='0' | mem_to_reg='0' (/tb_pipeline_debug/).
at 116 ns: Note: MEM | dado_mem=0000 | alu_result=0000 | reg_dest=010 (/tb_pipeline_debug/).
at 116 ns: Note: MEM | reg_write='1' | mem_to_reg='1' (/tb_pipeline_debug/).
at 116 ns: Note: WB | write_data=0000 | write_reg=010 | reg_write='1' (/tb_pipeline_debug/).

```

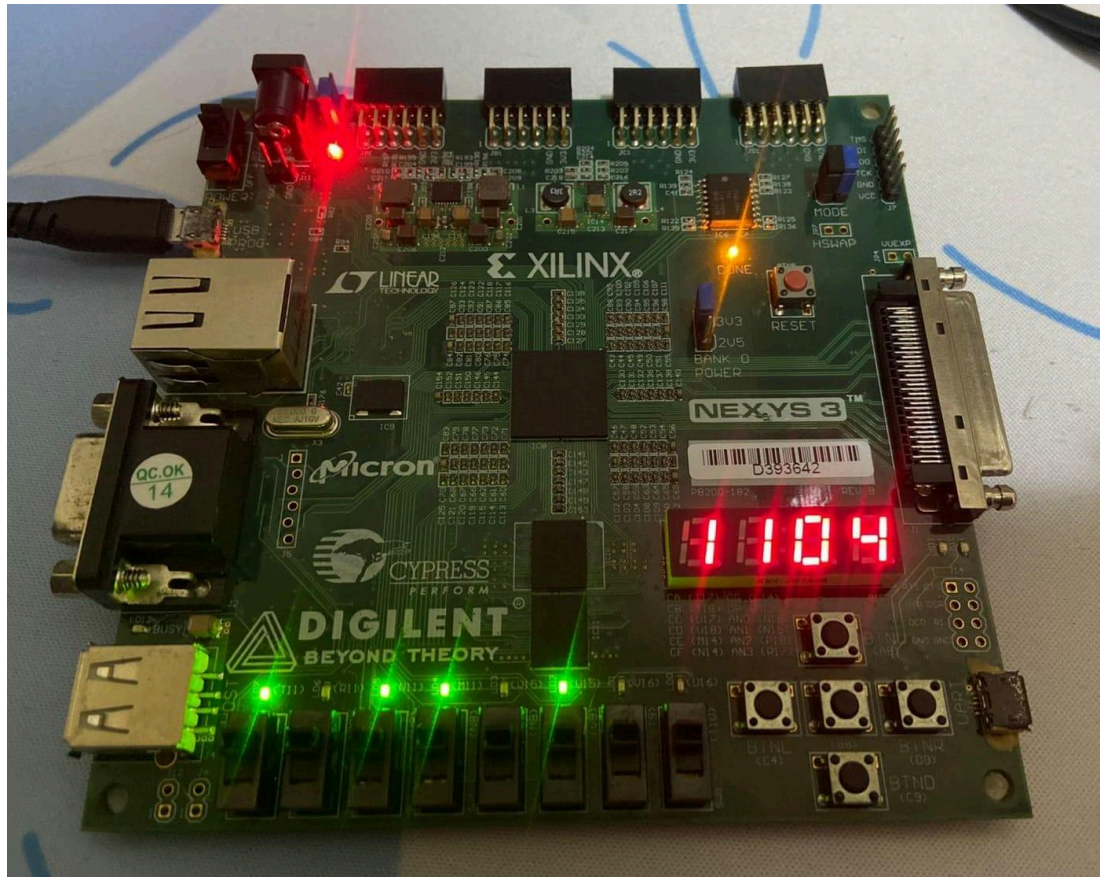
Assim, esse teste confirmou a limitação do pipeline implementado: instruções dependentes precisam ser separadas por NOPs, já que não há forwarding nem unidade de detecção de hazards para trata-los corretamente.

Conclusão dos Testes

Os testes realizados demonstraram o funcionamento correto das instruções implementadas no processador. As operações aritméticas e lógicas produziram os resultados esperados, as instruções de memória armazenaram e carregaram dados corretamente, e o teste misto confirmou a integração entre ambos os tipos.

Além disso, o teste de hazard demonstrou uma limitação esperada do projeto: como o processador não possui forwarding nem detecção automática de hazards, cabe ao programador inserir instruções NOP quando uma instrução depende diretamente do resultado de uma anterior.

Implementação em FPGA



Implementação do processador pipeline na placa FPGA Digilent Nexys 3

Após a validação por simulação, o processador foi sintetizado e implementado na placa FPGA Digilent Nexys 3. Durante os testes em hardware, o display de sete segmentos foi utilizado para exibir sinais internos do processador em tempo real.

Na Figura acima, o valor "11" exibido no display corresponde ao conteúdo atual do Program Counter (PC), indicando a posição da instrução que está sendo executada. Já o valor "04" corresponde ao sinal `wb_write_data`, representando o dado que está sendo escrito no banco de registradores durante o estágio Write-Back (WB).

Além disso, o botão central da placa foi utilizado como sinal de reset do processador, permitindo reinicializar a execução e retornar a arquitetura ao estado inicial para novos testes.

Conclusão

O projeto permitiu a implementação de uma arquitetura pipeline simplificada inspirada no processador MIPS, contemplando os cinco estágios clássicos de execução. Os resultados obtidos em simulação e em FPGA demonstraram o funcionamento correto das

instruções implementadas, bem como a transferência adequada de informações entre os registradores de pipeline. Além disso, a análise dos hazards permitiu compreender limitações típicas de arquiteturas pipeline sem mecanismos de forwarding ou detecção automática de dependências.